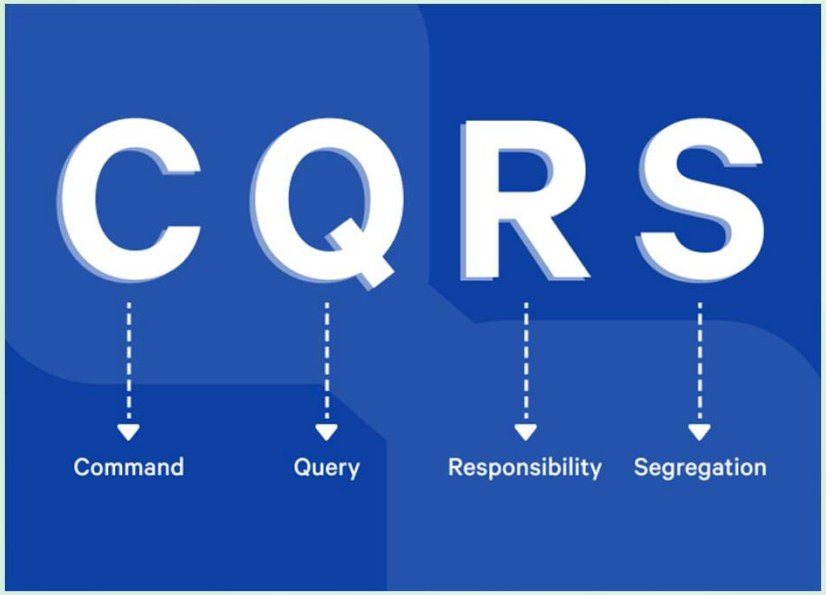
AWS DynamoDB ile CQRS View Uygulaması

Dersin Yürütücüsü: Dr. Göksel BİRİCİK

Ders: İleri Yazılım Mimarisi

Hazırlayan: Muhammed Kayra Bulut - 25501805

CQRS Nedir?



CQRS (Command Query Responsibility Segregation), Türkçe karşılığıyla Komut ve Sorgu Sorumluluklarının Ayrılması anlamına gelir. Bu kalıp, bir uygulamanın yazma işlemlerini (komutları) ve okuma işlemlerini (sorguları) iki farklı modelde ele almayı öneren bir mimari tasarım modelidir.

CQRS, **CQS (Command Query Separation)** ilkesinin bir uzantısı olarak kabul edilir.

CQS prensibi şu kurala dayanır:

- **Command (Komut)**: Nesnenin durumunu değiştirir, geriye değer döndürmez (void)
- **Query (Sorgu)**: Geriye değer döndürür, nesnenin durumunu değiştirmez

CQRS'in Temel Ayrımı

- **Write (Yazma) Operasyonları**: Veri ekleme, güncelleme, silme
- **Read (Okuma) Operasyonları**: Veri sorgulama ve görüntüleme

DynamoDB: Ölçeklenebilir NoSQL Çözümü

AWS DynamoDB, Amazon bulut ortamında hizmet olarak sunulan ölçeklenebilir bir NoSQL veritabanıdır. Veri modeli, JSON nesneleri gibi hiyerarşik ad-değer çiftlerinden oluşan öğeler içeren tablolardan oluşur.

DynamoDB tamamen yönetilen (fully managed) bir veritabanıdır ve bir tablonun işlem sınırını dinamik olarak artırıp azaltabilirsiniz.

Order History CQRS View Tasarımı

findOrderHistory() operasyonu için CQRS view, birden fazla servisten event tükettiği için bağımsız bir Order View Service olarak uyarlanmıştır.

OrderHistoryEventHandlers

Çeşitli servisler tarafından yayınlanan event'lere abone olur ve OrderHistoryDAO'yu çağırır

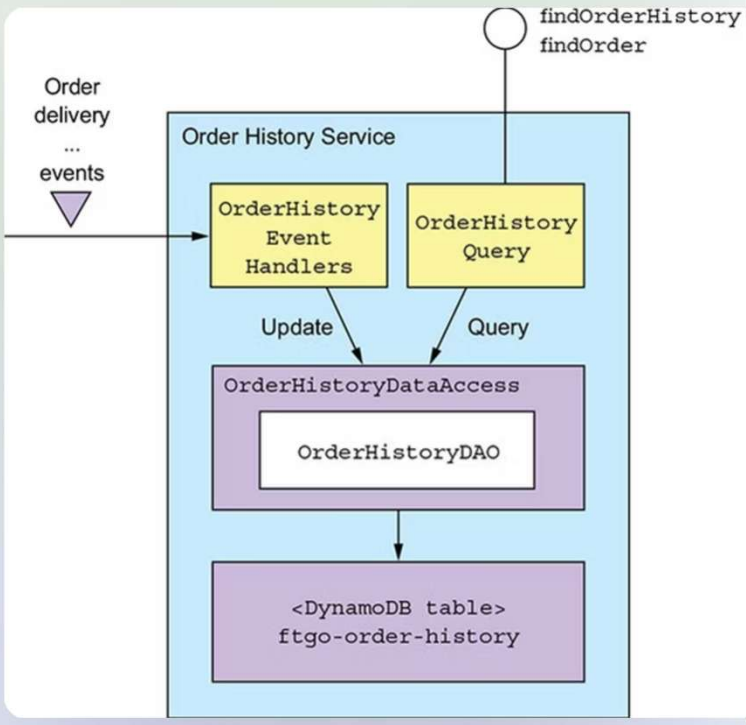
OrderHistoryQuery API

REST endpoint'lerini implemente eder ve sorgu operasyonlarını yönetir

OrderHistoryDataAccess

DynamoDB tablosunu güncelleyen ve sorgulayan OrderHistoryDAO metodlarını içerir

Bu sistemdeki sipariş bilgilerini depolamak için özel olarak tasarlanmış olan **ftgo-order-history** DynamoDB tablosu kullanılmaktadır. Bu tablo, sipariş geçmişine ait tüm verileri barındırır.



Event Handler Modülü

Yapı ve İşleyiş

OrderHistoryEventHandlers modülü, event'leri tüketen ve DynamoDB tablosunu güncelleyen event handler'lardan oluşur. Her handler metodu, event'ten türetilen parametrelerle OrderHistoryDao metodunu çağıran basit tek satırlık işlemlerdir.

Her event handler, event ve metadata içeren DomainEventEnvelope türünde tek bir parametre alır.

Önemli Metodlar

- **handleOrderCreated()**: OrderCreated event'ini işler ve veritabanında Order oluşturur
- **handleDeliveryPickedUp()**: DeliveryPickedUp event'ini işler ve Order durumunu günceller
- **makeSourceEvent()**: Event'i yayan aggregate'in tür ve ID'sini içeren SourceEvent oluşturur

DynamoDB Veri Modelleme Zorlukları

NoSQL veritabanları, RDBMS'lere (Relational Database Management System) kıyasla çok daha sınırlı veri erişim operasyonları sunar. Bu nedenle verilerin nasıl saklanacağını dikkatle tasarlamalısınız.

Tablo Tasarımı

ftgo-order-history tablosunun yapısının belirlenmesi

Index Tanımlama

findOrderHistory sorgusu için index oluşturulması

Sayfalama

Sorgu sonuçlarının sayfalanması

Order Güncellemesi

Order verilerinin güncellenmesi

Duplicate Detection

Tekrarlanan event'lerin tespiti

DynamoDB Primary Key ve Index Stratejisi

DynamoDB'de her tablo bir **primary key** gerektirir. Primary key, uygulamaların tablo öğelerini eklemek, güncellemek ve almak için kullandığı benzersiz tanımlayıcıdır.

1

Partition Key

DynamoDB'nin Z-axis ölçeklemesi, öğenin depolama partition'ını seçmek için partition key kullanır

2

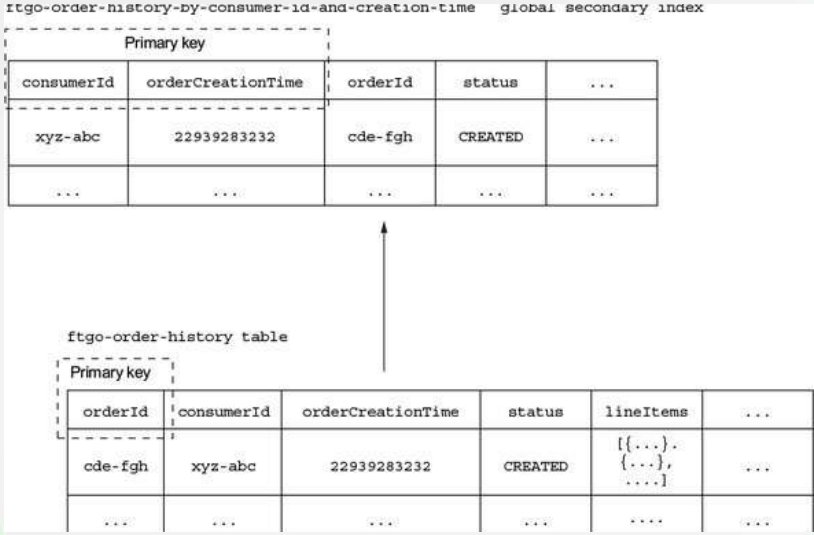
Sort Key

query() operasyonu, belirtilen sort key aralığındaki öğeleri döndürür ve sonuçları sort key'e göre sıralar

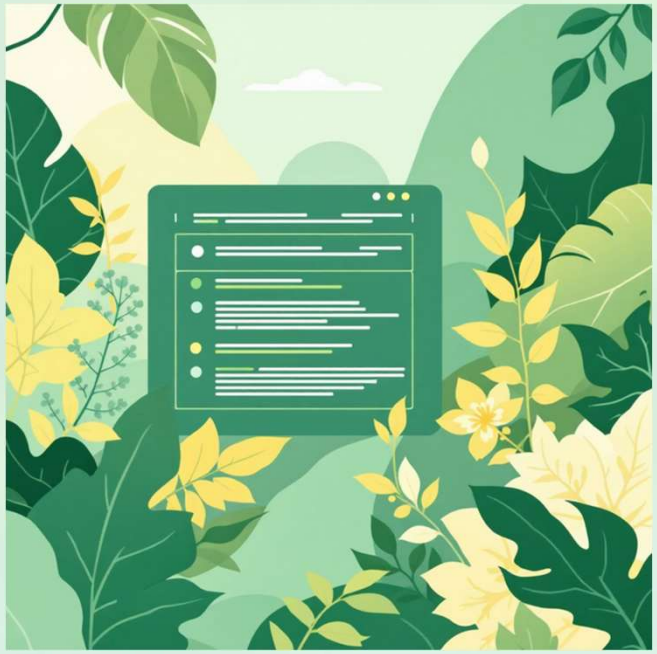
3

Secondary Index

consumerId ve orderCreationDate'i composite key olarak kullanan secondary index sorguları optimize eder



Sorgu İmplementasyonu ve Filtreleme



Filter Expression Kullanımı

findOrderHistory() query operasyonu, arama şartlarını belirten bir filter değişkenine sahiptir. DynamoDB Query operasyonu, **key condition expression** ile sort key üzerinde aralık kısıtlaması destekler.

Filter expression Boolean bir ifadedir ve DynamoDB Query yalnızca bu ifadeyi karşılayan öğeleri döndürür. Örneğin, CANCELLED olan Order'ları bulmak için: orderStatus = :orderStatus

Keyword Arama

Restaurant adı ve menü öğelerini tokenize ederek anahtar takımı (keyword set) oluşturur. contains() fonksiyonu ile eşleşen order'ları bulur.

Update Operasyonları ve Optimistic Locking

1

PutItem() vs UpdateItem()

PutItem() tüm öğeyi değiştirir, ancak eş zamanlı güncellemelerde sorun oluşturabilir. UpdateItem() ise öğenin sadece belirli özelliklerini günceller ve daha verimlidir.

2

Conditional Update Mekanizması

UpdateItem() operasyonu, condition expression true ise güncelleme yapar. Bu, tekrarlayan (duplicate) event'leri tespit edip atmak için kullanılır.

3

Event Tracking

Her öğe, onu güncelleyen event'leri kaydeder. «aggregateType»«aggregateId» özelliği, alınan en yüksek event ID'sini tutar. Event ID bu değerden küçük veya eşitse tekrarlı (duplicate) kabul edilir.

OrderHistoryDaoDynamoDb: Önemli Metodlar

addOrder() Metodu

ftgo-order-history tablosuna order ekler. İki değişken alır:

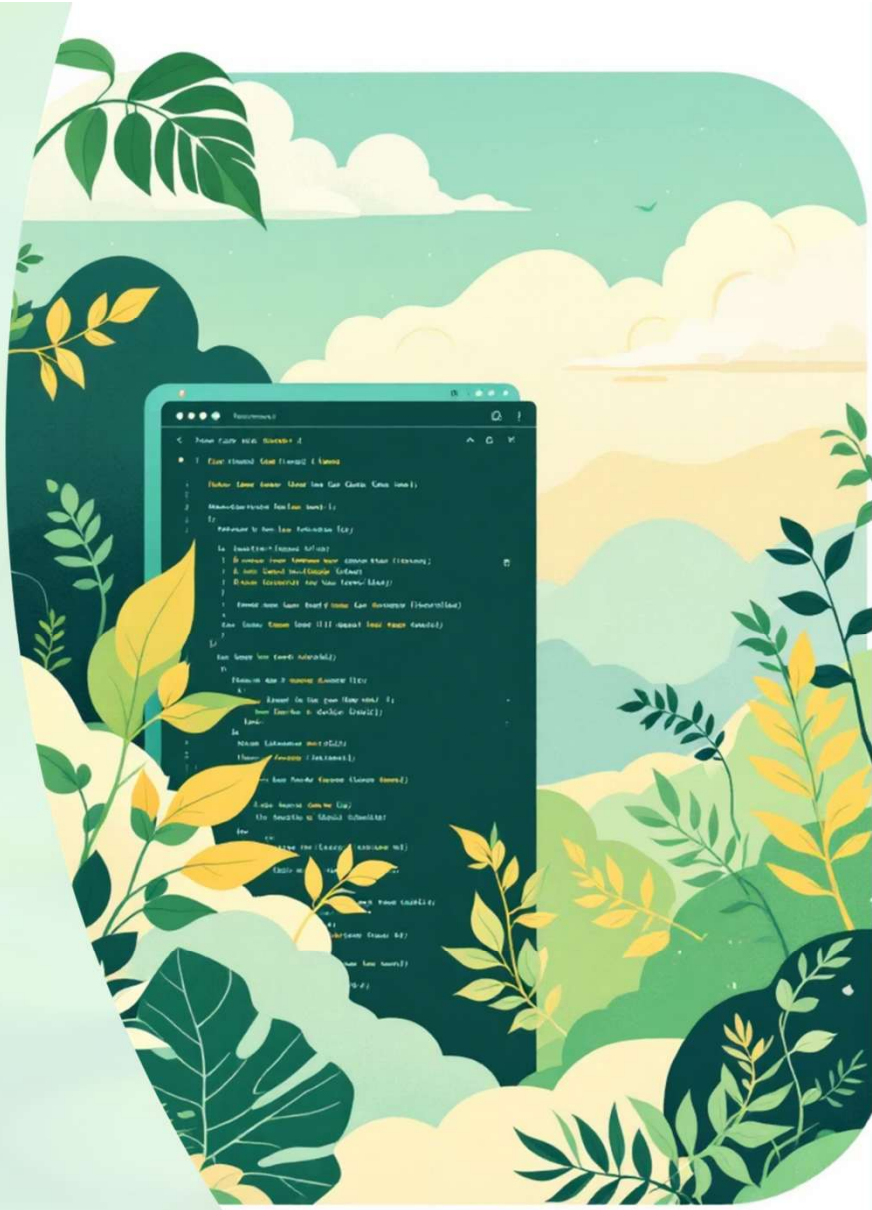
- **order**: OrderCreated event'inden elde edilen Order
- **sourceEvent**: eventId ve aggregate bilgilerini içerir

UpdateItemSpec oluşturur ve idempotent güncelleme için idempotentUpdate() metodunu çağırır.

notePickedUp() Metodu

DeliveryPickedUp event handler'ı tarafından çağrılır ve Order öğesinin deliveryStatus'unu PICKED_UP olarak değiştirir.

UpdateItemSpec oluşturur ve condition expression ekleyerek tekrarlı (duplicate) güncellemelere karşı koruma sağlar.



OrderHistoryDaoDynamoDb: Önemli Metodlar Devamı

idempotentUpdate() Metodu

Yinelenen event'lere karşı tablo güncellemelerini yönetir. SourceEvent.addDuplicateDetection() ile koşul ifadesi ekler. ConditionalCheckFailedException'ı yakalayarak mükerrer event'leri yok sayar ve veri tutarlılığını sağlar.

findOrderHistory() Metodu

Tüketici siparişlerini ikincil indexi kullanarak sorgular. consumerId ve filtre kriterlerine göre QuerySpec oluşturur, anahtar koşullarını ve filtre ifadelerini uygular. Sayfalama desteğiyle sonuçları OrderHistory nesnesine dönüştürür.

OrderHistoryDaoDynamoDb: findOrderHistory() Metodu

İkincil Index Sorgusu

findOrderHistory() metodu, tüketicinin siparişlerini almak için ftgo-order-history-by-consumer-id-and-creation-time ikincil indexini kullanır.

Parametreler ve Filtreleme

consumerId ve OrderHistoryFilter (arama kriterlerini belirten) olmak üzere iki ana parametre alır. QuerySpec oluşturarak sorguyu şekillendirir.

Sıralama ve Zaman Aralığı

withScanIndexForward(false) ile siparişleri artan yaşa göre (eskiden yeniye) sıralar. creationDate üzerinde RangeKeyCondition ile belirli bir zaman aralığındaki siparişleri getirir.

Gelişmiş Filtre İfadeleri

OrderHistoryFilter'dan gelen anahtar kelimeleri ve durum filtrelerini (örneğin CANCELLED siparişler) kullanarak dinamik filtre ifadeleri (keywordFilterExpression, statusFilterExpression) oluşturur.

Sayfalama Uygulaması

getLastEvaluatedKey() değerini JSON token'ına dönüştürerek sayfalama sağlar. Bir istemci start token belirtirse, withExclusiveStartKey() ile bu token'dan itibaren sonuçları getirir.

Sonuç: CQRS View Başarı Faktörleri

3

Ana Modül

Event handlers, Query API ve Data Access katmanları

100%

Idempotent

Tüm event handler'lar idempotent tasarlanmıştır

5

Tasarım Adımı

Tablo tasarımından pagination'a kadar kritik kararlar

CQRS view uygulaması, veritabanı seçimi, veri modelinin verimli tasarımı, eş zamanlı güncellemelerin yönetimi ve tekrarlı (duplicate) event'lerin ele alınması gibi çok sayıda sorunu çözmeyi gerektirir. En karmaşık kısım DAO katmanıdır, çünkü concurrency'yi doğru şekilde yönetmeli ve güncellemelerin idempotent olmasını sağlamalıdır.